

Go Fundamentals for Distributed Algorithms I

Danilo Kaćanski

Faculty of Technical Sciences, Novi Sad
Applied Algorithms in Control Systems

March 2026.

Getting Started with Go

What Kind of Language Is Go?

- Go is a **compiled, statically typed** programming language.
- It was designed for:
 - simplicity
 - readability
 - reliability
 - systems and network programming
- It has built-in support for **concurrency**.

Key Idea

Go is a practical systems language. It is simpler than C++, safer than C, and more structured for concurrent software.

Why Do We Use Go for Distributed Algorithms?

- The syntax is **small and easy to read**.
- Code compiles and runs **quickly**.
- It is well suited for:
 - message-passing systems
 - network services
 - concurrent protocols
- Many real distributed systems like **Kubernetes** and **Docker** are implemented in Go.

Motivation

We will use Go because it lets us focus on **distributed algorithms**, and not on the language complexity.

How We Will Use Go in This Course

- At the start we will use Go as a **tooling language**.
- After that, we will use Go to build a **simulator of distributed abstractions**.
- Later in the course, Go will also be used for **real concurrent implementations**.

Course Progression

Go foundations → distributed abstractions → $\dots$ → protocols and consensus

Basic Go Project Structure

Minimal Project Layout

```
my_project/  
  go.mod  
  main.go  
  util.go
```

- `main.go` is the entry point of the program
- `go.mod` is used for module definition and dependencies
- Also additional `.go` files can belong to the same package

Important

Go works with **packages**, not isolated files. Usually we run a project with `go run .` and not only `go run main.go`.

Minimal Go Program

Hello World

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, world!")
}
```

- `package main` defines an executable program
- `import` brings external packages into scope
- `func main()` is the entry point

Module Definition

Example go.mod

```
module my_project

go 1.22

require (
    github.com/some/lib v1.2.3
)
```

- Defines the **module name** (project identity)
- Specifies **Go version**
- Tracks **dependencies** and their versions
- Created with: `go mod init my_project`

Key Idea

`go.mod` makes builds **reproducible** and manages dependencies.

Organizing Code

Example util.go

```
package main

func add(a int, b int) int {
    return a + b
}
```

- Used to **split code into multiple files**
- All files share the same **package name**
- Functions in util.go can be used in main.go
- Improves **readability and organization**

Important

Go compiles all files in the same package **together**.

Language Basics

Primitive Types and Zero Values

- `string`, `int`, `bool`
- Every type has a **zero value**:
 - `string` → `""`
 - `int` → `0`
 - `bool` → `false`

Key Idea

Variables are always initialized so there are no undefined values.

Variable Declaration Styles

Three ways

```
// Explicit type  
var x int = 10
```

```
// Type inference  
var y = 10
```

```
// Short declaration (inside functions only)  
z := 10
```

- Explicit type can be used **anywhere** (including package level)
- Type can be **explicit** or **inferred**
- `:=` $\rightarrow$ short form, only inside func
- `:=` both **declares** and **assigns**

Integer Types and Aliases

- Signed: `int8`, `int16`, `int32`, `int64`
- Unsigned: `uint8`, `uint16`, `uint32`, `uint64`
- `byte` = `uint8` (alias)
- `rune` = `int32` (alias)

Note

`rune` is used for Unicode characters and only difference between it and `int32` is in intended use and convention.

Explicit Type Conversion

Example

```
var x uint8 = 10

// Explicit conversion required
var y int = int(x)
```

- Go does **not allow implicit conversions**
- Even between numeric types:
 - `uint8 int float64`
- Conversion must be **explicit and intentional**

What would fail

```
var y int = x // compilation error
```

Floating and Complex Numbers

Example

```
z := complex(3, 4)
r := real(z)
i := imag(z)
```

- float32, float64
- Complex numbers supported natively

Example

```
name := "DA"  
msg := "Hello " + name + "\n"
```

- Strings are text values
- Concatenation with +
- Escape sequences: \n

Typed vs Untyped Constants

Example

```
const x = 10 // untyped
const y int32 = 10 // typed

var a int = x // OK
var b float64 = x // OK

var c int = y // ERROR
```

- **Untyped constant:**
 - no fixed type
 - adapts to the context
- **Typed constant:**
 - fixed type (e.g. `int32`)
 - requires explicit conversion

Example

```
a := 5 + 3  
a += 2  
b := a << 1
```

- Arithmetic: + - * /
- Assignment: +=, ++
- Bitwise: |, <<, >>

Identifiers and Keywords

- **Identifiers** are names of variables, functions, types
- Rules:
 - Must start with a letter or `_`
 - Can contain letters, digits, `_`
 - Case-sensitive (`x` is different than `X`)
- **Export rule (important in Go):**
 - Uppercase → exported (public)
 - Lowercase → private
- **Keywords** are reserved: `func`, `var`, `const`, `package`, `import`, ...
- Numeric separators improve readability:

`10_000_000`

Runtime Error

- A **runtime error** occurs **while the program is executing**
- So the program is **compiled successfully**, but it fails during execution
- It is caused by invalid operations

Examples

- Dereferencing a nil pointer
- Writing to an uninitialized map
- Index out of bounds

Key Idea

Compile-time checks the code and **runtime** checks the execution.

What is panic?

- **panic** is Go's mechanism for handling runtime errors
- It is triggered when the program reaches an **invalid state**
- Execution stops and the program begins **stack unwinding**
- Panic propagates upward, defer runs during that process, and recover can stop it, else program **terminates**

Key Idea

panic = runtime error + immediate interruption of execution

Common Causes of panic

Nil map write

```
var m map[string]int  
m["a"] = 1 // panic
```

Nil pointer dereference

```
var p *int  
*p = 5 // panic
```

- Most runtime errors come from **uninitialized (nil) values**

Rule of Thumb

So if your program crashes, first thing you should do is to check for **nil**.

Exercise 1

Exercise 1

Task

Write a Go program that demonstrates core language basics.

- Declare variables:
 - `string`, `int`, `bool`, `float64`, `rune`
- Print their **values and types**
- Create at least:
 - one **untyped constant**
 - one **typed constant**
- Demonstrate:
 - **type conversion** (`int` $\rightarrow$ `float64`)
 - **string concatenation**
- Use one **bitwise operation** (`<<` or `|`)

Bonus

Show one example that would cause a **runtime panic** (comment it out).

Data Structures

Arrays: Fixed-Size Values

Array examples

```
var a [5]int // zero-value array: [0 0 0 0 0]
b := [5]int{1,2,3,4,5} // full initialization
c := [5]int{0:10, 3:40} // sparse init: [10 0 0 40 0]
d := [...]int{3,4,1,2} // compiler infers length [4]int
```

- Arrays always have **fixed length**
- Length is part of the type:

$[5]int \neq [6]int$

- Zero-value arrays contain zero values in all positions

Key Idea

An array is a fixed-size value, not a dynamic container.

Array Indexing and 2D Arrays

Indexing

```
b := [5]int{1, 2, 3, 4, 5}  
fmt.Println(b[0], b[4])
```

```
f := [2][2]int{{1, 2}, {3, 4}}  
fmt.Println(f[1][0])
```

- Indexing starts from 0
- Multi-dimensional arrays are arrays of arrays

Key Idea

Arrays are useful when the size is known and fixed in advance.

Arrays vs Slices

- **Array:**

- fixed size (`[5] int`)
- size is part of the type
- when one variable is assigned to another, all data is **just copied** and **not shared**
- stored directly in memory

- **Slice:**

- dynamic view over an array
- consists of: pointer + length + capacity
- when one variable is assigned to another, all data is **copied** and **shared**
- multiple slices can share the same underlying array

Important

Arrays copy data. Slices share data unless explicitly copied.

Key Idea

Array = data itself Slice = window into data

Slices: Dynamic Views

Slice basics

```
var a []int // nil slice: no underlying array; len=0, cap=0, a==nil  
  
b := []int{} // empty slice: has underlying array; len=0, cap=0, b!=nil  
  
c := make([]int, 3) // [0 0 0] len=3, cap=3  
  
d := make([]int, 3, 5) // [0 0 0] len=3, cap=5 (has extra space)
```

- **Nil slice** has no memory allocated yet
- **Empty slice** memory exists, but has no elements
- `make` allocates an underlying array

Key Idea

A slice is a **view** (**pointer + len + cap**) over an array, not the data itself.

Slice Length, Capacity, and append

append behavior

```
s := make([]int, 2, 3) // s = [0 0]; len=2 ; cap=3
fmt.Println(len(s), cap(s)) // 2 3
s = append(s, 10) // s = [0 0 10]; capacity not exceeded
s = append(s, 20) // capacity exceeded NEW array created, so s now points to a
different memory location
```

- **len** represents number of elements you can access
- **cap** is total space before reallocation
- **append**:
 - uses existing array if space exists
 - creates new array if capacity is exceeded

Key Idea

A slice may suddenly point to a **new array** after append.

Slicing and Shared Backing Arrays

Shared underlying array

```
a := []int{1, 2, 3, 4} // underlying array: [1 2 3 4]
x := a[1:3] // [2 3]
y := a[2:] // [3 4]
x[1] = 99 // modifies a[2]
fmt.Println(a) // [1 2 99 4]
fmt.Println(x) // [2 99]
fmt.Println(y) // [99 4]
```

- Slices point into the **same underlying array**
- x and y overlap because both of them include a[2]
- Modifying one slice changes the shared data

Ripple Effect

Slices are views into the same memory, not independent copies.

Controlling Sharing with Full Slice and copy

Limiting capacity with full slice

```
a := []int{1, 2, 3, 4} // full slice: low:high:max  
x := a[1:3:3] // len=2, cap=2 (cannot grow beyond index 3)  
dst := make([]int, len(x))  
copy(dst, x) // independent copy
```

- Full slice form: `a[low:high:max]`
- **low:high** decides what elements you will see
- **max** will limit slice capacity (how far it can grow)
- This prevents append from modifying the original array if needed

Key Idea

`copy()` creates fully independent data, so there will be no shared memory.

Strings, Bytes, and Runes

Unicode matters

```
s := "Go"
fmt.Println(s[0]) // byte
fmt.Println([]rune(s)) // Unicode-safe
```

- Strings are **immutable**
- Indexing a string returns a **byte**
- Use `[]rune` for Unicode-safe access
- Use `[]byte` for raw bytes

Note

Non-ASCII characters may use multiple bytes.

Maps: Key-Value Storage

Map basics

```
m := make(map[string]int)
m["alice"] = 10 // insert
m["alice"] = 15 // update
val := m["alice"] // 15
delete(m, "alice") // remove key
```

- Maps store **key-value pairs**
- Keys must be **comparable** (e.g., string, int)
- Reading a missing key returns the **zero value**
- Maps must be **initialized** before writing

Nil Maps and Reference Behavior

Nil map vs initialized map

```
var m map[string]int // nil map
fmt.Println(m["x"]) // OK returns 0
m["x"] = 10 // panic
n := make(map[string]int)
n["x"] = 10 // OK
```

- A **nil map** has no underlying storage
- Reading from a nil map returns the **zero value**
- Writing requires allocated memory, otherwise it will **panic**

Reference Behavior

Maps do not copy data, so multiple variables can refer to the same map.

Maps as Reference Types

Passing map to function

```
func update(m map[string]int) {  
    m["a"] = 100  
}  
  
func main() {  
    m := make(map[string]int)  
    update(m)  
    fmt.Println(m["a"]) // 100  
}
```

- Maps are **not copied** when passed to functions
- Function receives a reference to the same underlying data
- Modifications inside the function affect the original map

Key Idea

Maps behave like shared objects, so changes are visible everywhere, like in Python.

Structs: Custom Data Types

Student struct

```
type student struct {  
    firstName string  
    lastName string  
    age int  
    subjects []string  
}
```

- Structs group related fields
- Each field has its own zero value

Key Idea

Structs let us define our own data models.

Struct Initialization

Named vs positional initialization

```
s1 := student{  
    firstName: "John",  
    age: 20,  
}  
  
s2 := student{"Jane", "Smith", 22, nil}
```

- **Named initialization:**
 - fields explicitly specified
 - order does not matter
 - safer and more readable
- **Positional initialization:**
 - must follow exact field order
 - shorter, but error-prone

Anonymous Structs and Access

Anonymous struct

```
tmp := struct{ name string }{"Alice"}  
  
fmt.Println(tmp.name)
```

- **Anonymous struct:**
 - struct defined inline (no type name)
 - used for temporary, local data
- Fields are accessed using `.` operator
- Works like any other struct instance

Key Idea

Named structs are reusable types, anonymous structs are one-time data shapes.

Common Pitfalls

- Treating arrays and slices as the same thing
- Forgetting that slices may share the same backing array
- Assuming strings are mutable
- Writing to a nil map
- Forgetting that Unicode characters may need `[]rune`

Rule of Thumb

If data unexpectedly changes somewhere else, check whether you are sharing underlying memory.

Exercise 2

Task 1 — Arrays vs Slices

- Create an array: `[1, 2, 3, 4]`
- Create a slice from it (middle elements)
- Modify the slice
- Print both:
 - array
 - slice
- Then:
 - Create a copy using `copy()`
 - Modify the copy
 - Print results again

Goal

Understand sharing vs independent data.

Task 2 — Slice Capacity and append

- Create slice using `make`:
 - `length = 2`
 - `capacity = 3`
- Append two elements
- Print:
 - slice content
 - `length`
 - `capacity`
- Create another slice BEFORE append
- Modify both slices and observe behavior

Goal

Understand when `append` reuses memory vs allocates new array.

Task 3 — Maps and Structs

- Define a struct:
 - name (string)
 - age (int)

- Create a map:

```
map[string]student
```

- Add at least two entries
- Update one entry
- Delete one entry
- Try reading a non-existing key

Goal

Understand map behavior and struct usage.

Bonus Task — Slice Ripple Effect

- Create slice: `[1, 2, 3, 4, 5]`
- Create:
 - `a := s[1:4]`
 - `b := s[2:5]`
- Modify slice `a`
- Print:
 - original slice
 - `a`
 - `b`
- Explain the result

Goal

Understand shared memory and ripple effect.